

Fixed-Point Wordlength Allocation for ADMM Hardware Accelerators

Internal technical report — complete record including refuted hypotheses

6 September 2026

Abstract

We study fixed-point wordlength allocation for an ADMM solver implemented on FPGA fabric. The central observation is that a proximal operator annihilates quantisation error on its degenerate set: the fraction d of coordinates where the operator’s derivative vanishes contributes no error to the output, so the effective noise scales with $(1 - d)$ rather than the classical $q^2/12$ applied uniformly. Measured against bit-exact simulation the resulting model achieves ratios of 0.99 (L1) and 1.00 (Box); the classical model is wrong by $3\text{--}4\times$ typically and by $900\times$ at $d = 0.999$. We derive a range-feasibility bound on the penalty parameter, pre-registered and confirmed. On a second problem, LASSO, run unchanged on the same hardware, the model holds at 0.98–1.03 with d up to 0.77 and an off-lattice threshold. On routed Artix-7 silicon, narrowing the main datapath from Q2.16 to Q2.9 reduces LUTs by 46.6%, raises $F_{\max}$ by 28.8%, and cuts dynamic power from 11 mW to 7 mW, giving $184 \rightarrow 117$ nJ per solve. We report a negative result: per-operator asymmetric lane allocation costs 4.3% area at both main widths with no power or frequency compensation. All hardware figures are measured — SAIF-based power, swept $F_{\max}$, gate-level bit-exact validation — not estimated, and both designs run bit-exact on a board at their in-context $F_{\max}$ (65 and 82.5 MHz), where the speed-up is +26.9%. We state explicitly where the model does not hold: it predicts ensemble-mean behaviour and under-predicts per-instance error by up to 7.6 bits.

Contents

1	Introduction and scope	3
2	Related work and precise positioning	3
2.1	The primary prior art	3
2.2	The second invitation	3
2.3	The gap, confirmed by the incumbents	3
2.4	Comparisons that must not be made naively	4
3	Error model	4
3.1	Degenerate-set annihilation	4
3.2	Range feasibility	4
3.3	Loop-gain relation	5
3.4	Ordering of lane precision needs	5
4	Fabric cost model	6
5	Profitability of asymmetric allocation — a negative result	6

6	Hardware results	7
6.1	Method	7
6.2	All seven builds	7
6.3	Headline	7
6.4	Throughput	7
6.5	On the board, at speed, in context	7
6.6	Two critical-path regimes	8
7	A second problem: LASSO	8
8	Comparison with prior accelerators	9
9	Measurement discipline	10
10	Hypotheses tested and refuted	11
10.1	Composition (Theorem 5 as originally planned)	11
10.2	Active-set churn	11
10.3	F_{MAIN} dominance	11
11	Scope limits	11
11.1	The model predicts ensemble means, not instances	11
11.2	Other limits	12
12	Reproducibility	12
13	Outstanding	13

1 Introduction and scope

An accelerator for a proximal splitting method must choose a fixed-point format for every signal in its datapath. Too few fraction bits and the solver returns a wrong answer; too many and the fabric cost, critical path and switching energy all grow. The classical treatment models every quantisation as an independent additive noise source of power $q^2/12$ and propagates it through the algorithm. This report argues that treatment is substantially wrong for proximal operators, quantifies by how much, and validates the correction on silicon.

What is claimed. An error model in which the proximal operator’s degenerate set annihilates a measurable fraction of injected quantisation noise; a pre-registered range-feasibility bound; a measured demonstration that per-operator asymmetric allocation is net-negative in fabric; and a fully measured hardware characterisation of seven configurations.

What is not claimed. This is not the first error model for fixed-point ADMM — Jerez et al. have one. It is not a robustness bound. It does not supply a mechanism for every empirical relation reported. Section 8 lists the hypotheses that were tested and failed; Section 9 lists the scope limits.

2 Related work and precise positioning

2.1 The primary prior art

Jerez, Goulart, Richter, Constantinides, Kerrigan and Morari (arXiv:1303.1090) implement fixed-point ADMM on FPGA with an explicit round-off error analysis: a two-step error recurrence, Schur stability via Lyapunov argument, and a converging upper bound. They state the bound determines a priori the minimum number of bits for a given accuracy specification. **Any claim of a first error model for fixed-point ADMM is unavailable.**

The opening they leave is specific and is the basis of this work. In deriving their bound they observe that for the projection step no arithmetic error is introduced, and that the error *can only be reduced by multiplication with a diagonal matrix with entries componentwise in $[0, 1]$* . They then set that matrix to the identity to keep the bound conservative. That discarded diagonal is exactly the degenerate-set annihilation quantified here.

2.2 The second invitation

Kinsman and Nicolici (TCAD 30(9), 2011) allocate bit-widths for iterative computations by SMT, with conjugate gradient as their case study — the nearest published problem to ours. Their quantisation constraints, they write, *assume no correlation between quantisation error and value, when in reality correlation may exist. If the nature of the correlation is known, it can be captured into constraints.*

Their paper has sixteen citations in fifteen years and none extends it to iterative optimisation solvers; the descendants went toward range and overflow analysis rather than precision models for nonlinear operators.

2.3 The gap, confirmed by the incumbents

Ha and Sentieys state in both DATE 2020 and DATE 2023 that existing analytical techniques are *limited to modeling noise power metrics of Linear and Time-Invariant (LTI) systems (with some extensions)*. A proximal operator is not LTI. Two papers named the gap; a third confirms it was still open in 2023.

2.4 Comparisons that must not be made naively

Wu, Wallace, Mota et al. (J. Signal Process. Syst. 2022) report up to 67% LUT and 60% power reduction for mixed-precision ADMM. Those figures are *not* comparable to ours. Their baseline is FP32 floating point, so most of the gain is the float-to-fixed conversion itself; ours is Q2.16 fixed narrowed to Q2.9, with the float-to-fixed gain already banked in the baseline. Their power is estimated with the Xilinx Power Estimator; ours is measured from post-route switching activity. Their precision axis is temporal (per iteration); ours is spatial (per operator lane).

Usefully, their own Table 1 corroborates our negative result: for the ADMM solver in fixed point, consistent FXP24 uses 6775 LUTs while mixed FXP20→24 uses 6807 — non-uniform precision costing area, unremarked.

3 Error model

3.1 Degenerate-set annihilation

Let P_θ be a proximal operator with parameter θ , evaluated at fractional wordlength W with step $q = 2^{-W}$. Partition the input coordinates into the degenerate set D , where $\partial P/\partial v = 0$, and its complement.

Theorem 1 (Operator error decomposition).

$$\mathbb{E}\|e_P\|^2 = \frac{|D^c|}{n} \left[L^2 \frac{q^2}{12} + \sigma_{\theta, D^c}^2 \right] + \frac{|D|}{n} \sigma_{\theta, D}^2 \quad (1)$$

Writing $d = |D|/n$, the classical model overestimates by roughly $1/(1-d)$. On the degenerate set the operator’s output does not depend on the input, so injected perturbation is annihilated rather than propagated.

Validation. `model/error_decomp.py` measures the operator in isolation on a v -distribution harvested from float ADMM runs.

operator	model/measured	classical/measured	d range
L1 soft-threshold	0.99	3–4×	0.00–0.84
Box projection	1.00	3–4×	0.00–0.86
L2 ridge scaling	0.88	—	0

Table 1: Theorem 1 against bit-exact simulation. At $d = 0.999$ the classical model is wrong by 900×

[SCOPED: L2 gives 0.88, a 12% miss. L2 has $d = 0$, so the annihilation term is inactive and the discrepancy lies entirely in the θ term — the γ coefficient quantisation, which Corollary 1a states is not attenuated. The annihilation mechanism is validated; the coefficient-quantisation magnitude is not, to better than 12%. Quoting “0.99–1.00” without naming the operators would be selective.]

3.2 Range feasibility

Proposition 1 (Penalty lower bound). *Weight representability requires $\rho \geq 2^{-(I-1)}$ for I integer bits.*

This was registered before measurement. With $I = 2$ it predicts $\rho \geq 0.5$. Measured: $\max |M| = 2.116$ at $\rho = 0.125$ (infeasible in Q2.x), 1.117 at $\rho = 0.5$ (feasible). The pre-registration is the point and should be stated as such.

3.3 Loop-gain relation

Operator-level error does not determine solver-level error; the ADMM loop amplifies it by a gain G . For the contractive L2 lane,

$$\log G = a \log R + c, \quad R = \|(I - T)^{-1}\|, \quad (2)$$

with R the resolvent of the linearised iteration. Fitted on one set of problem instances and tested on a *disjoint* set, the error is 0.061 bits in-sample and **0.046 bits held out**, inside even the 0.17-bit bar Li et al. set for their SQNR model. Per instance, the model under-predicts by at most +0.66 bits at the 99th percentile (+0.73 worst observed), so adding one bit to its prediction covers 99% of instances.

[REFUTED: For L1 and Box, fitted forms $\log G = a \log \frac{1}{1-\|M\|_2} + b R_{\text{op}} + c$ with $R_{\text{L1}} = d$, $R_{\text{Box}} = d^2$ achieved 0.340 and 0.234 bits under leave-one-condition-out validation, but on the *same* instances. On disjoint instances they reach 1.38 and 0.73 bits.] For L1 the ensemble mean is dominated by a single instance (68% of the ensemble error in the median cell) and shifts 0.59 bits between instance sets — more than the error the fit claimed. Box’s held-out error is 4.7 times its own instance-to-instance variability, so it is over-fitted rather than noisy. A robust (geometric-mean) statistic does not rescue L1 (0.87 bits), and is ill-posed for Box, where some instances have exactly zero error. For these two lanes only qualitative behaviour is claimed — loop gain grows with conditioning and tracks the resolvent — together with the zero-parameter ordering predictor of Section 3.4. No mechanism is claimed for either; the active-set churn hypothesis was tested and refuted (Section 8.2).

The accuracy criterion. Li et al. hold their model to < 1 dB, about 0.17 bits. The like-for-like comparison is Theorem 1, an analytical noise model against bit-exact simulation, and it meets that bar: 0.007, 0.000 and 0.092 bits for L1, Box and L2. For loop-level claims we use 0.5 bits, for three reasons: it is the resolution of an integer width decision; no criterion can be tighter than the quantity’s own repeatability, measured here as 0.075 (L2), 0.156 (Box) and 0.593 (L1) bits between instance sets; and it is applied to held-out instances only.

3.4 Ordering of lane precision needs

Which operator needs the most lane precision is not fixed. Continuous W^* over a conditioning sweep shows the Box–L2 gap moving from -0.33 bits at $\kappa(H^\top H) = 2$ to $+2.33$ at 1000, crossing between 10 and 20.

The operator coefficient alone does not predict this: it gets the sign wrong in 6 of 8 resolvable Box–L2 cases and all 14 L1–L2 cases, missing by 1.2–1.5 bits. Multiplying by the loop resolvent $R = \|(I - T)^{-1}\|$, computed from M with no fitted parameter,

$$\Delta W_{ab} = \frac{1}{2} \log_2 \frac{A_a R_a}{A_b R_b}, \quad (3)$$

gives 0.288 bits and 8/8 signs for Box–L2 and places the crossover near 20. L2’s contraction pins its resolvent ($2.1 \rightarrow 3.2$) while L1 and Box grow ($2.7 \rightarrow 200$), so the contractive lane becomes relatively cheaper as the problem hardens.

[SCOPED: L1–L2 is 0.621 bits and misses the two smallest gaps in sign. L1–Box is under-predicted by 0.4–0.6 bits and the predictor is blind to κ ; the omitted κ -quantisation term of Corollary 1a is the likely cause and is untested. The exponents fitted in the loop-gain model make Box–L2 *worse* (1.05 bits, 4/8): the zero-parameter resolvent generalises across problem generators, the fitted model does not.]

4 Fabric cost model

Theorem 2 (Lane area). $A_{\text{lane}}(W) = bW + c$, with $b \approx 19.6$ Slice LUT/bit.

The metric matters, so it is stated: b is fitted on Slice LUTs from `report_utilization`. On LUT cell counts the same fit gives $b = 26.4$ and misses the held-out width by -15.5% rather than -6.3% (`model/lane_area.py`).

[REFUTED: The originally proposed quadratic form $A(W) = aW^2 + bW + c$ fails its held-out test.] Fitting on the three no-cast points (lane width = F_{MAIN} : $16 \rightarrow 269.9$, $10 \rightarrow 159.0$, $9 \rightarrow 129.0$ LUT/lane) and predicting lane 8 (actual 122.0):

form	fit	predicts $W=8$	error
quadratic	exact (3 pts, 3 params)	95.7	-21.6%
linear	3.0% max residual	114.3	-6.3%

The fitted a is negative (-1.646): the measured slope *falls* from 30 LUT/bit ($9 \rightarrow 10$) to 18.5 ($10 \rightarrow 16$). A fabric-multiplier area law must be convex.

Resolution. The 166 vs 35 LUT measurement supporting a quadratic law was taken on the γ multiplier *in isolation*, where it does hold (4.74 measured against 4 predicted). At whole-lane granularity the multiplier is a small share — L1 and Box contain no multiplier at all — so casts, comparisons and saturating adds dominate and the total is linear. The W^2 term is not refuted; it is undetectable at lane granularity on this design. Both laws may be quoted provided the granularity is stated.

Cast cost, measured directly. Two builds share lane width 8 but differ in cast depth: 122.0 LUT/lane at $F_{\text{MAIN}} = 10$ against 127.5 at $F_{\text{MAIN}} = 16$, giving $c_{\text{cast}} \approx 0.9$ LUT/bit/lane.

5 Profitability of asymmetric allocation — a negative result

Theorem 3 (Profitability). *Asymmetric lane allocation is profitable only if area saved exceeds cast overhead. With lane area linear, saving is $b\Delta W$ — linear and small — so cast overhead dominates and asymmetry is net-negative.*

Note this derivation now rests on the linear law, which survived its falsifier, rather than on the rejected quadratic one.

matched pair (same F_{MAIN} , one lane narrowed)	LUT	F_{max}
asym_derived vs uniform10 ($F_{\text{MAIN}}=16$)	+4.3%	-5.5%
main9_asym vs main9_uniform ($F_{\text{MAIN}}=9$)	+4.3%	+1.3%

The area penalty is $+4.3\%$ at both widths, identical to one decimal place — a reproducible cost rather than two coincidentally similar numbers. Dynamic power is identical across three measured pairs. The F_{max} effect is inconsistent in sign and the $+1.3\%$ is one binary-search step, so no frequency claim is made in either direction.

Statement: per-operator asymmetry costs 4.3% area at both main-datapath widths, buys no measurable dynamic power, and has no consistent frequency effect.

6 Hardware results

6.1 Method

Artix-7 XC7A35T, out-of-context synthesis. Power from post-route gate-level simulation writing SAIF, consumed by `report_power`; every run Confidence High with 72–75% of nets matched, and bit-exact against the golden model in the same run that produced the activity. $F_{\max}$ by binary search on the clock constraint, seven full implementations per build, reported as 1000/(tightest period that met timing) — never 1000/(period – slack).

6.2 All seven builds

build	F_{MAIN}	lanes	LUT	$F_{\max}$ (MHz)	dyn (mW)	total (mW)
uniform18	16	16/16/16	6308	66.3	11	80
uniform10	16	8/8/8	4439	67.6	10	78
asym_derived	16	8/7/8	4631	63.9	10	78
main10_uniform	10	10/10/10	4081	79.7	7	76
main10_lane8	10	8/8/8	3665	68.9	7	75
main9_uniform	9	9/9/9	3366	85.4	7	76
main9_asym	9	9/8/9	3510	86.5	7	76

6.3 Headline

uniform18 $\rightarrow$ main9_uniform: LUT -46.6% , $F_{\max} +28.8\%$ (bracketed $+27.6\%$ to $+30.4\%$ by the search resolution), dynamic power 11 $\rightarrow$ 7 mW, energy per solve 184 $\rightarrow$ 117 nJ.

Report absolutes, not the percentage. Power reports round to 1 mW, so 7 and 11 carry ± 0.5 mW and the honest bound on the reduction is 29–44%. Energy per solve is the better figure: dynamic energy per cycle is CV^2 and therefore frequency-independent, so -36% holds regardless of the operating clock a reader imagines.

Static power dominates. Device static is 68 mW of the 76–80 mW total and is identical across builds. Total on-chip power falls only 5.0%. **Any power claim must say *dynamic* explicitly or it is wrong by a factor of seven.**

6.4 Throughput

Cycles = $26K + 3$ exactly, across all seven builds, four main widths, symmetric and asymmetric. Cycles per iteration are invariant to every width parameter, so all performance gain comes from $F_{\max}$.

[SCOPED: Iteration counts are *not* comparable across widths. With $\varepsilon = 0$ the early exit fires when the residual quantises to exactly zero, which happens sooner at coarser precision — stagnation, not convergence. main9 needs 15/19/12 iterations against uniform18’s 28/32/22; computing solutions per second from each build’s own iteration count bakes in a spurious $1.8\times$. Hold iterations fixed.]

6.5 On the board, at speed, in context

Every figure above is out-of-context: the core alone, no I/O, no clock network. To check that the headline survives a real design, both headline builds were wrapped for a Basys3 board (ROM-fed problems, UART results), each clocked by one MMCM at an exact frequency, and

swept downward in 2.5 MHz steps (1.25 MHz for `uniform18`). A bitstream was written only for the highest frequency meeting setup *and* hold after routing, in context; the board’s output was then diffed against the golden vectors over UART.

build	$F_{\max}$, OOC	in context: met / missed	on the board
<code>uniform18</code>	66.3 MHz	65 / 66.25 MHz	3 operators bit-exact, 65 MHz
<code>main9_uniform</code>	85.4 MHz	82.5 / 85 MHz	3 operators bit-exact, 82.5 MHz

The wrapper and clock network cost 2–3.4% at the met point. The speed-up in context is +26.9% at the met points, bracketed +24.5% to +30.8% by the step size, against +28.8% (+27.6% to +30.4%) out of context. The brackets overlap: the frequency headline is a property of the datapath, not of the out-of-context methodology. Energy per solve is frequency-independent and unaffected.

A methodological trap, recorded. The initial bring-up used a fabric clock divider behind a LUT multiplexer. Put through the same in-context timing gate it fails by 5.7 ns setup and 2.6 ns hold: the multiplexer places both clocks on one net, and the LUT-routed clock carries enough skew to break hold across the design. The GUI flow writes such a bitstream without complaint, and it produced correct results at 50 MHz — functional evidence, but not a timing-clean claim. The results above are from the MMCM builds only.

6.6 Two critical-path regimes

Worst-path endpoints split the builds into two families with no overlap:

build	worst path endpoint	levels	$F_{\max}$
<code>uniform18, uniform10, asym_derived, main10_lane8</code>	prox lane <code>z.out_reg</code>	21–24	63.9–68.9
<code>main10_uniform, main9_uniform, main9_asym</code>	<code>converged_reg</code>	16–17	79.7–86.5

$F_{\max} = 1 / \max(\text{prox path}, \text{convergence-test path})$. Narrowing a lane below F_{MAIN} inserts a narrowing cast into the prox path; when that lifts it above the convergence-test path the build drops into the slow regime. `main10_lane8` and `main10_uniform` isolate this to one variable: same F_{MAIN} , same main datapath, different lane width, different binding path, 11 MHz apart.

Design rule: narrowing the main datapath raises $F_{\max}$ only while the convergence-test path binds. Once a lane cast pushes the prox path above it, further narrowing buys area and nothing else.

Within-family spread is $\approx 8\%$ and is implementation variance. The family names the primal or dual residual interchangeably across builds; both feed `converged_reg` through the same comparison logic, so this is the *convergence-test* path and must not be described as a specific residual.

7 A second problem: LASSO

Every result above comes from one problem family. LASSO, $\min_x \frac{1}{2} \|Ax - b\|^2 + \lambda \|x\|_1$, maps onto the *unchanged* datapath: $M = (A^\top A + \rho I)^{-1}$, $q = A^\top b$, and the L1 lane with $\kappa = \lambda / \rho$. We use $N = 8$, a 2-sparse ground truth, Gaussian A with m rows and 20 dB SNR. Four predictions were registered in `model/lasso.py` before it was run.

The test is harder than it looks. On the MIMO workload the L1 lane has $d \approx 0$, so Theorem 1’s $(1 - d)$ factor had been exercised for L1 only by sweeping κ with κ placed on the lattice. LASSO yields $d = 0.62\text{--}0.77$ natively, with κ wherever λ puts it.

ID	prediction	result	
P-L1	T1/meas $\in [0.9, 1.1]$, native distribution	0.981–1.028 (5λ)	pass
P-L2	$m=32$: $F^* \leq 9$; $m=6$: $F^* > 9$	8; 10–11	pass
P-L3	support at F^* matches double $\geq 99\%$	90–100% per cell	fail
P-L4	bit-exact on LASSO vectors	RTL at $F_{\text{MAIN}} = 16, 9$; board at 9	pass

P-L1. At $\lambda/\lambda_{\text{max}} = 0.05, 0.1, 0.2, 0.3, 0.5$ the ratio of Theorem 1’s prediction to measured lane error is 0.988, 1.028, 1.019, 0.981, 0.993 (60 instances, $m = 16$, $F = 6\text{--}12$). The classical $q^2/12$ model is off by $1.6\text{--}2.7\times$. That is less than $1/(1 - d)$ ($2.7\text{--}4.4$) because κ is off-lattice and its quantization adds error that Theorem 1’s θ -term includes and the classical model does not.

P-L2. Width need follows $\|M\|_2 \rightarrow 1/\rho$ as rows are removed: $F^* = 8$ at $m = 32$ ($\|M\|_2 = 0.74$), 9 at $m = 16$ (0.87), 9–11 at $m = 8$ (0.99), 10–11 at $m = 6$ (1.00); no saturation in any cell. This is a trend over twelve cells, not a fitted law, and F^* is not monotone for $m \leq 8$.

P-L3 failed. Support agreement with double precision at F^* is 90–100% per cell, not $\geq 99\%$. *Post hoc* (`lasso.py --e3`), all 11 disagreeing coefficients are ≤ 2 LSB of F^* in magnitude (82% ≤ 1 LSB): threshold ties at the resolution of the format. This characterises the failure; it does not rescue the prediction.

P-L4. Three instances ($m = 32, 16, 8$) run bit-exact through the unmodified RTL at $F_{\text{MAIN}} = 16$ and 9, and on the Basys3 board at $F_{\text{MAIN}} = 9$ from a single bitstream holding all three. The $m = 8$ instance recovers the wrong support relative to ground truth in double precision as well; every comparison in this section is against double-precision LASSO on the same instance, never against x_0 .

8 Comparison with prior accelerators

Read the caveats before the numbers. Every row below solves a different problem, at a different size, on a different device, to a different and mostly unstated accuracy. The table is therefore not a ranking, and this paper claims no efficiency advantage over any row in it. We compute no normalised metric (LUT-seconds, energy per variable) precisely because the accuracy at which each work’s solve time was obtained is not stated often enough to make one meaningful. The table is generated by `tools/comparison_table.py` from `data/comparison_lit.json`, where every entry carries a source URL and a record of who verified it.

work	device / format	problem	LUT	MHz	power (method)
This work (baseline)	Artix-7 35T, 18 b (Q2.16)	N=8	4673	66.3	11 mW (SAIF, routed)
This work	Artix-7 35T, 11 b (Q2.9)	N=8	2574	85.4	7 mW (SAIF, routed)
Jerez 2014	Virtex-6, 18 fb	216 var	n/r	400	n/r (—)
ADMIN 2021	V7 690T, 18 b	16x16 det.	13392	263	n/r (—)
TASER 2016	V7 690T, 14 b	N=9 det.	4790	232	600 mW (tool est.) †
Wu 2022	ZCU106, mixed 20->24 b	n=16 patch	6807	403	307 mW (XPE)
Hamadouche 2023	ZCU106 HLS, 24 b	LASSO, 5 it.	1822	157	44 mW (post-impl. est.)
RSQP 2023	Alveo U50, fp32	$10^1\text{--}10^5$ var	24245	300	19 W (board meas.)
Zhang 2025	ZCU102, ap.fixed 24/9	1619 var	147238	250	10.7 W (unstated)
AccelMPC 2026	Artix-7 100T, 32 b	H=40, k=10	n/r	100	n/r (post-route est.)
Peccin 2020	MAX10 (DE10-Lite), 16 b (10 frac)	Nu=5 GPC	n/r	50	n/r (—)

† forward–backward splitting, not ADMM.

What the table actually shows. Three things, none of them a speed claim.

Wordlength is assumed, not derived. Of the fixed-point rows, every one states a single wordlength as a given — 18 bits, 14 bits, 24 bits, 32 bits — with no account of where it came from. Jerez et al. are the exception in kind, offering a bound rather than a chosen number, and their own empirical table stops at 18 fraction bits. This paper’s 11 bits is derived from a model and then confirmed on routed silicon.

Power is usually estimated. Two rows report no power at all; three report vendor or post-implementation estimates; one reports a card-level measurement of a datacentre part; one states a figure without a method. Our 7 mW is a post-route figure annotated with switching activity from a gate-level simulation of the actual regression vectors, at Vivado *Confidence High*. We showed earlier that the vectorless number for the same build is $\sim 10\times$ too high, which is the size of error these methodologies differ by.

Nobody reports the full set. No prior ADMM accelerator we found reports resources, activity-based power, a met $F_{\max}$, and a per-solve time together for a fixed-point design. That, rather than any single figure, is where this paper’s measurement discipline differs.

Fairness decisions, stated rather than hidden. TASER is forward-backward splitting, not ADMM, and is marked with a dagger; it is kept because it is the same group, part and fixed-point discipline as ADMIN. Peccin et al. report Intel logic elements (30 662 LE, 62% of a MAX10), which are not Xilinx Slice LUTs, so their area cell is left empty rather than converted — but their number format is the most directly comparable choice in the table: 16 bits with 10 fractional bits, stated without derivation, on a low-cost board, which is exactly the decision this paper’s model replaces. AccelMPC reports utilisation only as percentages and gives no absolute power; dividing its published energy by its published latency would imply a few hundred milliwatts, but that quantity is ours, not theirs, so it is recorded in the data file and not in this table. Escarate et al. (ICA-ACCA 2022) implement ADMM on a ZCU104 through Vitis HLS and report execution times only (0.365–1.567 ms for 1–50 iterations), calling their own figures a reference floor at default settings; with no resources, no power and no stated number format, the row would be empty and they are cited here instead.

Two rows that must be read carefully. Wu et al.’s headline (67% LUT, 60% power) is against an FP32 baseline; ours is Q2.16 fixed narrowed to Q2.9, with the float-to-fixed win already in the baseline. And our own row is the smallest problem on the smallest part: $N = 8$ on a \$100 Artix-7 board, not a ZCU106 or an Alveo. A reader comparing absolute LUTs across those parts is comparing fabrics, not designs.

9 Measurement discipline

Three claims were withdrawn when vectorless estimates were replaced by measurement, and they are recorded because the near-miss is instructive.

1. Vectorless power was $\approx 10\times$ too high (112 vs 11 mW; 78 vs 7 mW). The ratio survived roughly intact by luck.
2. **[REFUTED: “Asymmetric pruning cuts dynamic power 18.8%”]** — measured, both configurations give 11 mW. The difference was an artefact of the estimator.
3. $F_{\max}$ of 71.1 MHz was $1000/(20\text{ ns} - \text{slack})$ at a deliberately loose constraint. Swept properly it is 85.4 MHz.

The near-miss. Comparing a vectorless `uniform18` (112 mW) against a measured `main9` (7 mW) would have produced a fabricated 94% power reduction. It was caught only because an

object-count guard in the SAIF flow printed zero. Every number in Section 6 is measured, and the closest competing work estimates its power with a spreadsheet tool.

10 Hypotheses tested and refuted

Recorded because each would have read plausibly in a submission draft.

10.1 Composition (Theorem 5 as originally planned)

[REFUTED: 2.509 bits mean error against a 0.5-bit criterion.] The original theory placed contraction at the operator. The falsifier showed it is not there; contraction lives at the loop. The resolvent saturates at 3.18 with $g = 0.667$ against divergence to 47,669 at $g = 1$. This negative result produced the loop-gain reframing and was predicted in advance as the claim most likely to fail.

10.2 Active-set churn

The L1/Box coefficient sign difference invited a mechanism: L1’s degenerate set churns (entries crossing the soft threshold) while Box’s freezes (entries pinned to a clip bound). Churn was measured directly as the fraction of coordinates changing degenerate-set membership per iteration.

[REFUTED: Two of three predictions fail.] L1 churn averages 0.0011; Box averages 0.0028 — Box churns *more*, and both are $\approx 0.1\%$, i.e. both essentially frozen. Churn also fits worse than d (LOO 0.781 against 0.340 for L1). No mechanism is available and none is claimed.

10.3 F_{MAIN} dominance

An earlier claim that narrowing the main datapath dominates lane width on all three axes was carried by a confound: `uniform10` and `main10_uniform` differ in both parameters. The control build `main10_lane8` was added and **[REFUTED: the frequency half fails]** — holding lanes at 8, narrowing F_{MAIN} from 16 to 10 buys $+1.9\%$ F_{max} , while *widening* lanes to match F_{MAIN} buys $+15.7\%$. On area both axes are real and lanes are better per bit (-5.1% /bit against -2.9% /bit). The correct account is the two-regime mechanism of Section 6.5.

11 Scope limits

11.1 The model predicts ensemble means, not instances

`model/adversarial.py` attacks the fitted loop-gain model using Kinsman and Nicolici’s own construction — right-hand sides aligned across the eigenvectors of the system matrix — plus a greedy search maximising model error. Errors are reported signed, because the sign is the whole story: positive means the model *under*-predicts error, so wordlengths chosen from it would be too few.

operator	worst under-prediction, random rhs	adversarial
L1	+7.58 bits	+11.43 bits
Box	+2.16 bits	+3.43 bits
L2	+0.60 bits	+2.25 bits

[SCOPED: Random instances already break the 0.5-bit criterion.] The per-instance distribution on held-out instances (signed; + means under-prediction):

operator	p50	p90	p95	p99	max
L2	-0.09	+0.41	+0.53	+0.66	+0.73
Box	-0.92	+1.67	+2.76	+3.46	+4.05
L1	-3.06	+0.60	+1.13	+1.90	+4.22

[SCOPED: Only L2 yields a design rule (one extra bit). The L1 and Box quantiles are margins on models that do not generalise, so they are not one.]

What this does not invalidate. The Q2.9 design point was chosen with model guidance and then verified bit-exact against the golden model on routed silicon. The model proposes; measurement disposes. That division of labour is the methodology and should be stated, not hidden.

Why not a bound instead. Converting Theorem 1 into a guaranteed bound requires the Lyapunov or interval machinery that Jerez et al. and Kinsman and Nicolici already possess. Their bounds are conservative by construction — hundreds of bits for conjugate gradient, reducing to roughly single-to-double precision only after heavy input-space restriction — which is the deficiency a predictive model exists to address. Competing there means losing on their ground with their tools. For L1 and Box, per-instance safety therefore comes from simulation — which is how Q2.9 was chosen and verified — or from the prior art’s bounds at their cost in conservatism.

11.2 Other limits

- Two LUT metrics appear in this report and must not be mixed. The -46.6% headline counts LUT *cells* (`get_cells REF_NAME =~ LUT*`): $6308 \rightarrow 3366$. Section 8 quotes Slice LUTs from `report_utilization`, which is what other papers report and what LUT combining makes smaller: $4673 \rightarrow 2574$, -44.9% . Both come from the same routed checkpoints; the ratio survives either way.
- Power reports round to 1 mW; this is the binding constraint on every power claim, including the apparent saturation at $F_{\text{MAIN}} = 10$.
- $N = 8$, one problem family, one device. No claim is made about scaling.
- The L1–Box ordering residual (0.4–0.6 bits, and a κ trend of -2.04 bits per unit) is unexplained. Parameter-quantisation terms and an outlier explanation were both tested and refuted.

12 Reproducibility

No number in this report exists without a script that produces it.

artefact	produced by
Theorem 1 validation	model/error_decomp.py
Loop-gain fit	model/loop_gain.py, loop_gain_l1.py
Churn refutation	model/box_form.py
Composition refutation	model/generalize.py
Adversarial scope test	model/adversarial.py
LASSO (P-L1–P-L3)	model/lasso.py
LASSO vectors (P-L4)	model/gen_vectors_lasso.py
Comparison table	tools/comparison_table.py
Board builds, in context	syn/board_build.tcl, tools/board_table.py
Routed resource counts	tools/util_table.py
Golden vectors	model/gen_vectors_cfg.py
Builds	syn/run_ooc.tcl
SAIF power	syn/saif_power.bat, saif_log.tcl
$F_{\max}$ sweep	syn/fmax_sweep.tcl
Critical-path location	syn/critpath.tcl
Literature pass	tools/lit_sweep.py

13 Outstanding

- Measured wall power (blocked on instrumentation).